Reviewer Pack — Coxeter Polynomial Root Count Inverse Proportional to Resolu…

Entry 102ae91c53d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 21:50:22 UTC

Coxeter Polynomial Root Count Inverse Proportional to Resolution Length in 3-SAT

Entry ID: `102ae91c53d3` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-10 21:50:22 UTC

1. Conjecture

Field A (mathematical branch): Coxeter Group Representation Theory **Field B** (complexity object): Resolution Proof Length

Statement:

> For a 3-SAT instance φ, let C_φ be its associated Coxeter group element derived via clause-to-reflection mapping. The number of distinct roots of the Coxeter polynomial Π_φ(x) satisfies |roots(Π_φ)| ≥ $\log_2(\rho(\varphi)) + 1$, where ρ(φ) is the resolution proof length of φ.

Rationale (proposer's reasoning):

> Coxeter polynomials encode symmetry invariants of reflection groups, which may mirror structural constraints in clause interaction graphs. These invariants could expose hidden algebraic regularities that govern proof complexity trade-offs.

Taxonomy category: `LIFTING` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `cf7c3f0923717782`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    m, n = len(A), len(b)
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, m):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(m - 1, -1, -1):
        x[i] = Fraction(b[i], A[i][i])
        for j in range(i - 1, -1, -1):
            b[j] -= A[j][i] * x[i]
    return x

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
```

```
    det = Fraction(0)
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        sign = (-1) ** (j % 2)
        det += sign * A[0][j] * determinant(submatrix)
    return det

def coxeter_polynomial(C):
    m, n = len(C), len(C[0])
    identity = [[Fraction(1 if i == j else 0) for j in range(n)] for i in range(m)]
    C_inv = gaussian_elimination(identity, [C[i][j] for j in range(n) for i in range(m)])
    det = determinant(C_inv)
    return det

def clause_to_reflection(clause):
    n = len(clause)
    reflection = [[Fraction(0) for _ in range(n)] for _ in range(n)]
    for var in clause:
        if var > 0:
            reflection[var-1][var-1] = Fraction(-2)
            reflection[0][var-1] = Fraction(1)
            reflection[var-1][0] = Fraction(1)
        else:
            reflection[-var-1][-var-1] = Fraction(-2)
            reflection[0][-var-1] = Fraction(1)
            reflection[-var-1][0] = Fraction(1)
    return reflection

def dpll_with_clause_learning(clauses):
    n = max(abs(var) for clause in clauses for var in clause)
    assignment = [None] * (n + 1)
    learned_clauses = []

    def is_satisfiable():
        stack = []
        while stack or any(c for c in clauses if all(assignment[abs(v)] == v > 0 for v in c)):
            while not stack and any(c for c in clauses if all(assignment[abs(v)] == v > 0 for v in
                clause = next(c for c in clauses if all(assignment[abs(v)] == v > 0 for v in c))
                learned_clauses.append(clause)
                var = next(v for v in clause if assignment[v] is None)
                stack.append((var, True))
            if not stack:
                return False
            var, polarity = stack.pop()
            assignment[var] = -var if polarity else var
            for c in learned_clauses:
                if all(assignment[abs(v)] == v > 0 for v in c):
                    learned_clauses.remove(c)
        return True

    if is_satisfiable():
        return len(learned_clauses)
    else:
        return None

def run_trial(seed: int) -> dict:
```

```
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    clauses = []
    for _ in range(n):
        clause = [random.randint(1, n), -random.randint(1, n), random.randint(1, n)]
        random.shuffle(clause)
        clauses.append(clause)

    C = [[Fraction(0) for _ in range(n)] for _ in range(n)]
    for clause in clauses:
        C += clause_to_reflection(clause)

    resolution_length = dpll_with_clause_learning(clauses)
    if resolution_length is None:
        return {
            "metric_name": "resolution_length",
            "metric_value": 0,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "unsatisfiable"
        }

    coxeter_poly = coxeter_polynomial(C)
    roots = [root for root in range(-10, 11) if abs(coxeter_poly.subs(x=root)) < 1e-6]
    num_roots = len(roots)

    conjecture_holds = num_roots >= math.log2(resolution_length) + 1
    counterexample = "" if conjecture_holds else f"resolution_length={resolution_length}, roots={r

    return {
        "metric_name": "number_of_roots",
        "metric_value": num_roots,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
```

```
        counterexample_desc = results[next(i for i, r in enumerate(results) if not r["conjecture_h
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e3581c5.py", line 152, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e3581c5.py", line 120, in run_trial
    C += clause_to_reflection(clause)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7e3581c5.py", line 77, in clause_to_ref
    reflection[-var-1][-var-1] = Fraction(-2)
    ~~~~~~~~~~^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed during clause-to-reflection mapping, preventing evaluation of conjecture validity | next: Fix the IndexError in clause_to_reflection by verifying reflection matrix dimensions

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107885
2	propose	ollama_remote	qwen3:8b	0	0	111426
3	propose	ollama_remote	qwen3:8b	0	0	78962
4	propose	ollama_remote	qwen3:8b	0	0	79333
5	propose	ollama_remote	qwen3:8b	0	0	42853
6	preregistration	ollama_remote	qwen3:8b	0	0	24157
7	novelty	ollama_remote	qwen3:8b	0	0	21053
8	novelty	ollama_remote	qwen3:8b	0	0	16429
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22245
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18299
11	judge	ollama_remote	qwen3:8b	0	0	14701

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 537343 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/102ae91c53d3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/102ae91c53d3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/102ae91c53d3.tar.gz` (if generated)